

Week 10

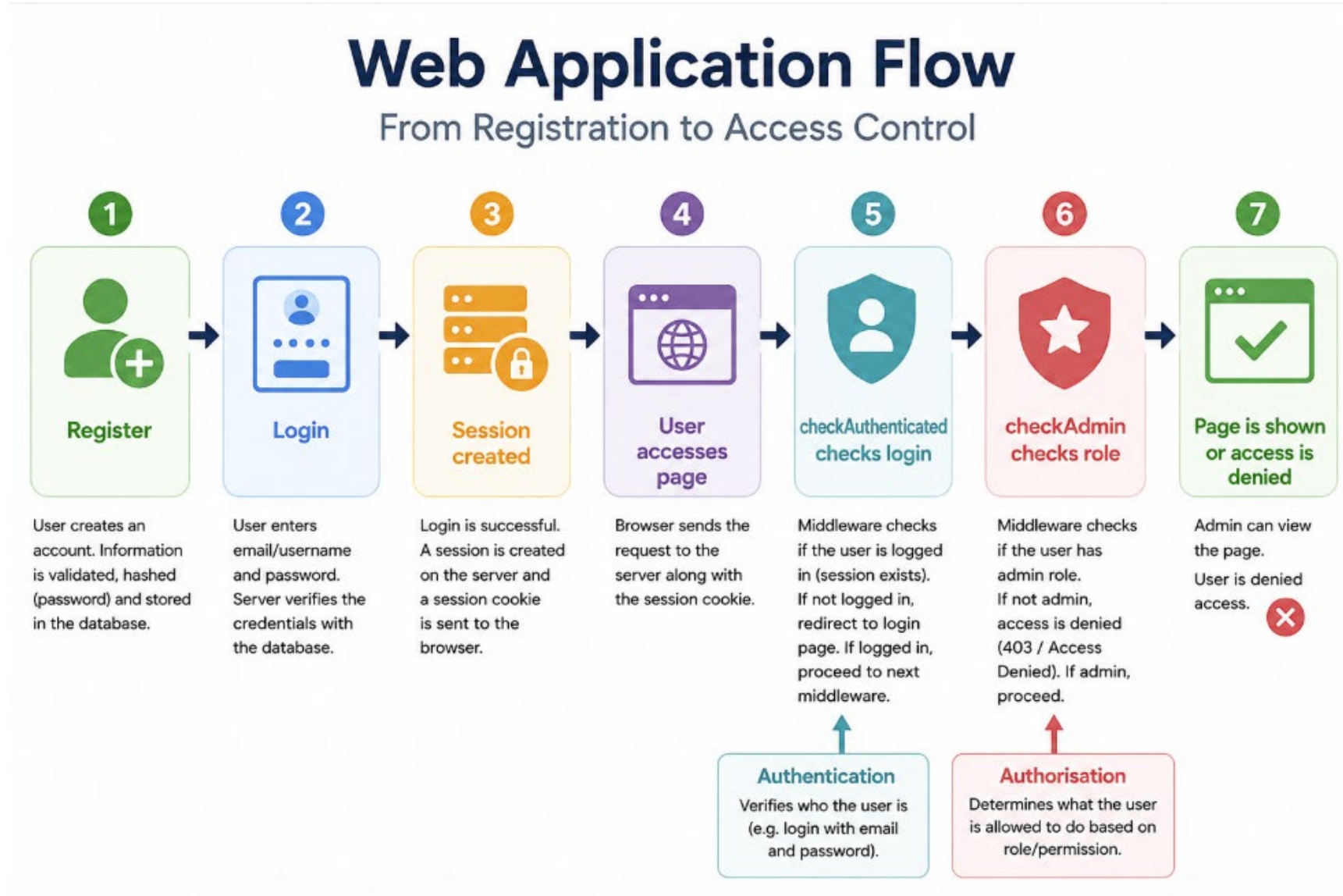
C237:

Software Application Development

LESSON 19:

Authentication and Authorisation

Overview





Registration

Registration

- Registration is the process of **creating a new user account** in the application.
- A registration typically includes:
 - Name
 - Email / Username
 - Password
 - Role (Optional, e.g. user or admin)

What Happens During Registration?

- User fills in a form
- Server validates the input
 - E.g.
 - Valid Data (e.g. proper email format, password length)
 - Unique (e.g. no duplicate email)
- **If okay:**
 - Password is **hashed**
 - Data is **stored in the database**
 - User **redirected to Login Page**
- **If not okay:**
 - User gets an **error message**

Registration isn't just filling in a form - it triggers a series of server-side actions.

Let's Try!

- Create a folder L19 in your C237 folder.
- Copy the folder **registrationApp** found in your “L19 Activity Files” folder and paste it into the L19 folder which you created earlier.
- Import your database in **MySQL Workbench** using the “**C237_usersdb.sql**” file found in your “L19 Activity Files” folder.

[illegible]

Let's Try!

- Open the registrationApp (the one in your L19 folder) in Visual Studio Code.
- Open terminal and **install** the **required packages**:
 - **npm init -y**
 - **npm install express**
 - **npm install ejs**
 - **npm install mysql2**
 - **npm install connect-flash**:
 - a middleware for Express applications that shows **temporary messages** (e.g. Login Error, Logout Success).
 - **npm install express-session**:
 - a middleware for **managing sessions** in Express applications. Sessions allow you to store login info (session data) between page visits.



Express Session

Why do we need Sessions?

- HTTP requests are independent. Without sessions, the server does not automatically remember that a user has logged in.
- A session **allows the server to remember user information** across different page visits.

Set up express-session

- Open **app.js** (RegistrationApp) in Visual studio code and add in the code below:

```
const express = require('express');
const mysql = require('mysql2');

//TO DO: Insert code to import 'express-session'
const session = require('express-session');

const flash = require('connect-flash');

const app = express();
```

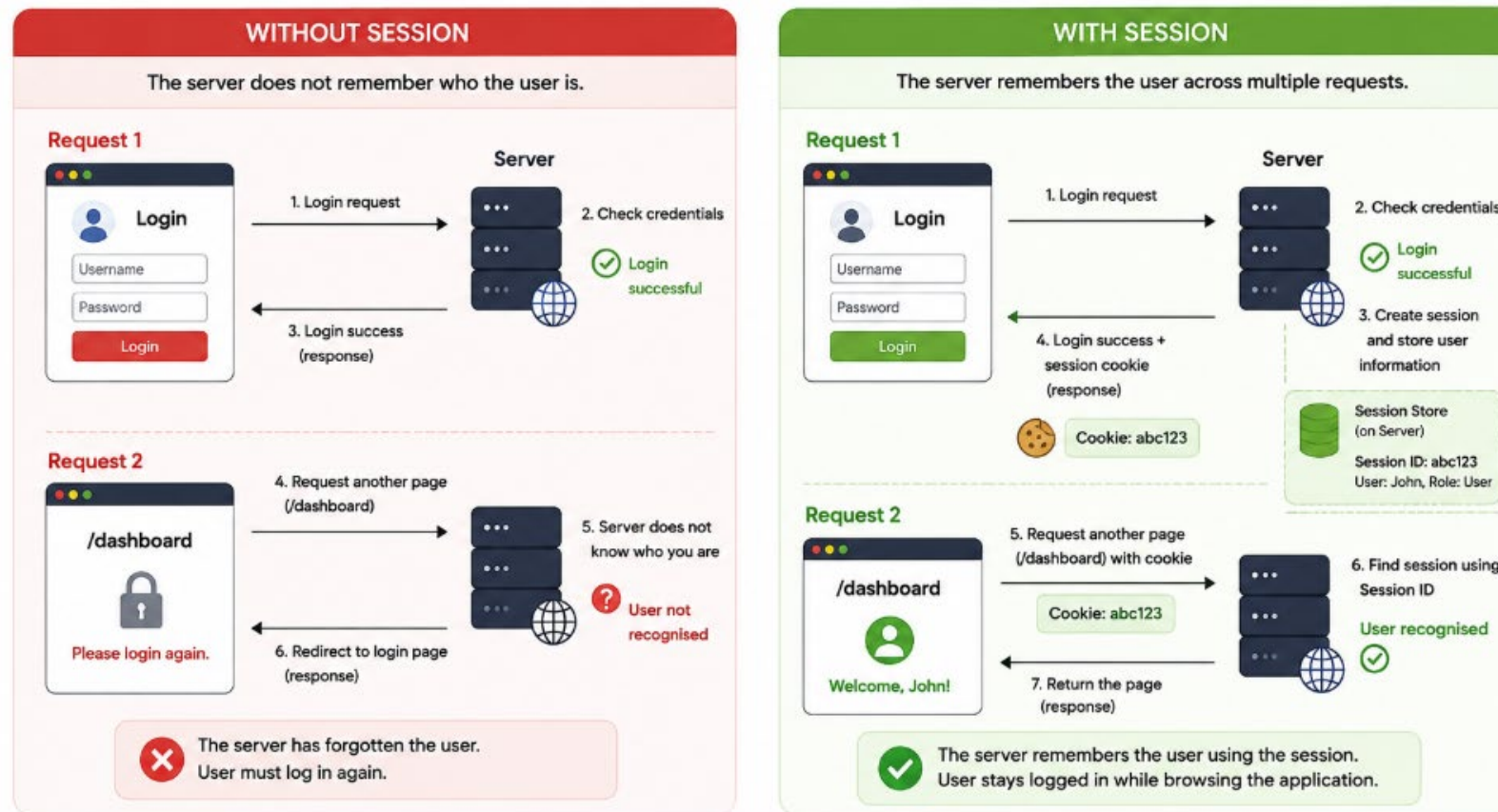
```
app.use(express.urlencoded({ extended: false }));
app.use(express.static('public'));

//TO DO: Insert code for Session Middleware below
app.use(session({
  secret: 'secret',
  resave: false,
  saveUninitialized: true,
  // Session expires after 1 week of inactivity
  cookie: {maxAge: 1000 * 60 * 60 * 24 * 7}
}));
```

1000 → milliseconds in 1 second
60 → seconds in 1 minute
60 → minutes in 1 hour
24 → hours in 1 day
7 → number of days

Why do we need Sessions?

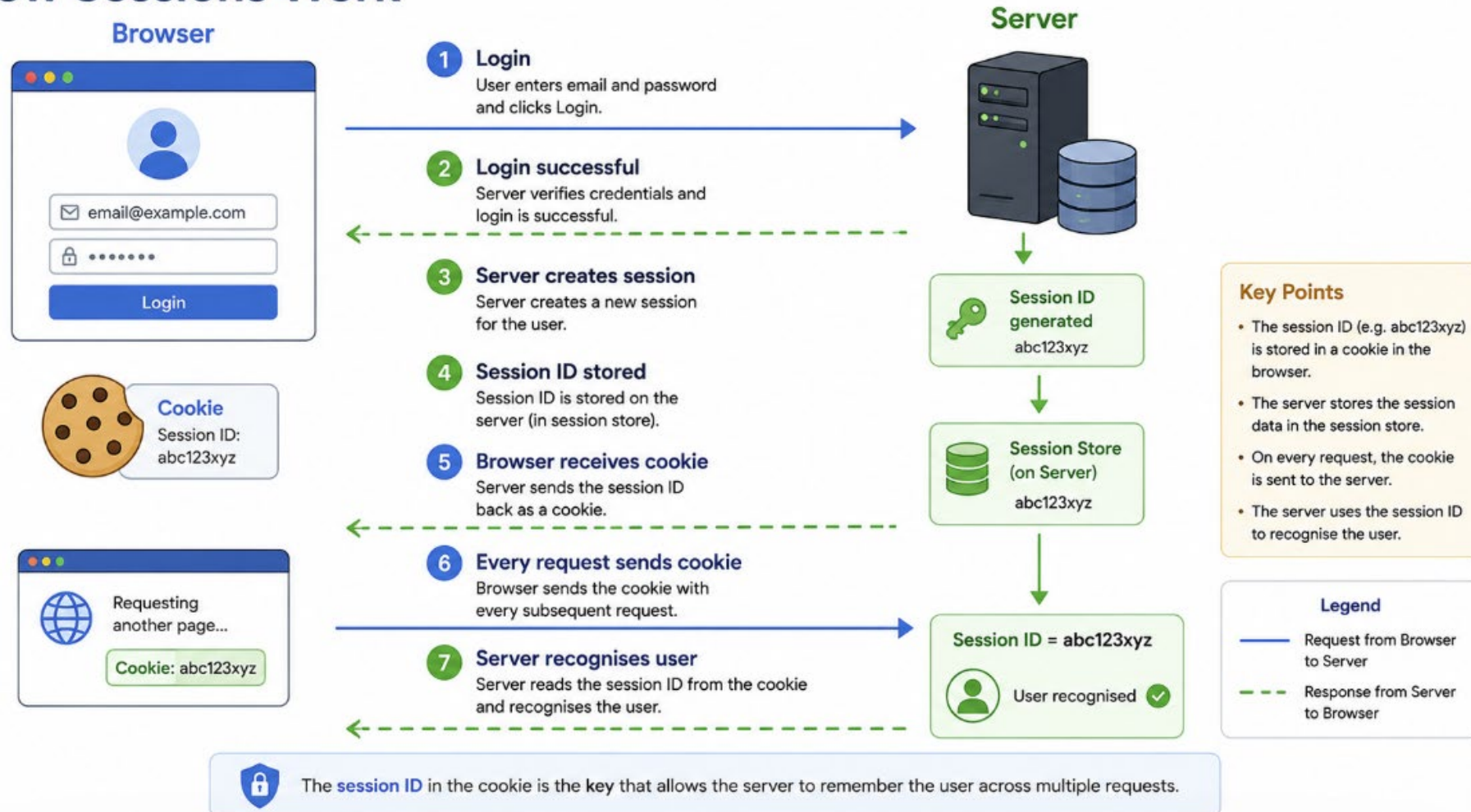
- HTTP is stateless. Without sessions, the server forgets the user between requests.



Key Idea: A session allows the server to remember who the user is after they have logged in.

Session Middleware – How it works

How Sessions Work



Server-Side Validation

- Add **server-side validation** to an existing **registration form**.
- Steps:
 1. Create a **middleware function** `validateForm`.
 2. **Integrate** `validateForm` into the **registration route**.
 3. **Test** the validation by submitting various forms.

Step 1: middleware function

- Open **app.js** (RegistrationApp) in Visual studio code and add in the code below:

```
/** ***** TODO: Create a middleware function validateRegistration ***** **//
const validateRegistration = (req, res, next) => {
  const { username, email, password, address, contact } = req.body;

  if (!username || !email || !password || !address || !contact) {
    return res.send('All fields are required.');
```

This Validation function

- Checks that all required fields are filled
- Ensures password is at least 6 characters
- If not valid, shows an error or redirects user

```
  }

  if (password.length < 6) {
    req.flash('error', 'Password should be at least 6 or more characters long');
    req.flash('formData', req.body);
    return res.redirect('/register');
  }
  //If all validations pass, the next function is called, allowing the request to proceed to the
  //next middleware function or route handler.
  next();
};
```

Step 2: Integrate function into route

- Open **app.js** (RegistrationApp) in Visual studio code and add in the code below:

```
/******* TO DO: Integrate validateRegistration into the register route *****/
app.post('/register', validateRegistration, (req, res) => {

  const { username, email, password, address, contact } = req.body;

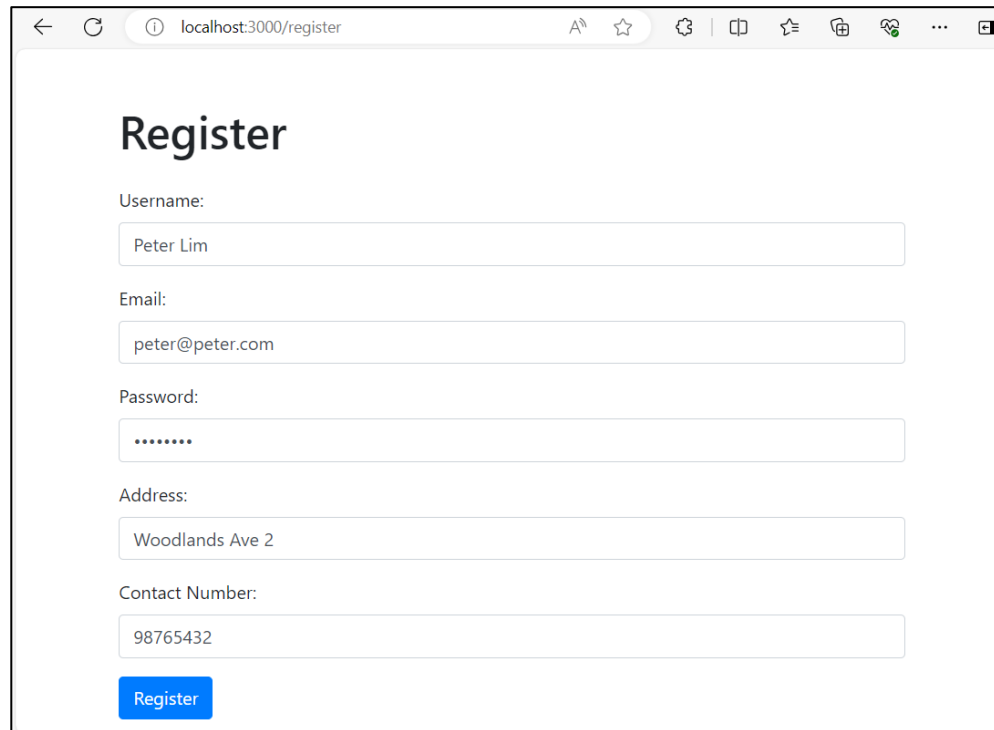
  const sql = 'INSERT INTO users (username, email, password, address, contact) VALUES (?, ?, SHA1(?), ?, ?)';
  db.query(sql, [username, email, password, address, contact], (err, result) => {
    if (err) {
      throw err;
    }
    console.log(result);
    req.flash('success', 'Registration successful! Please log in.');
```

```
    res.redirect('/login');
  });
});
```

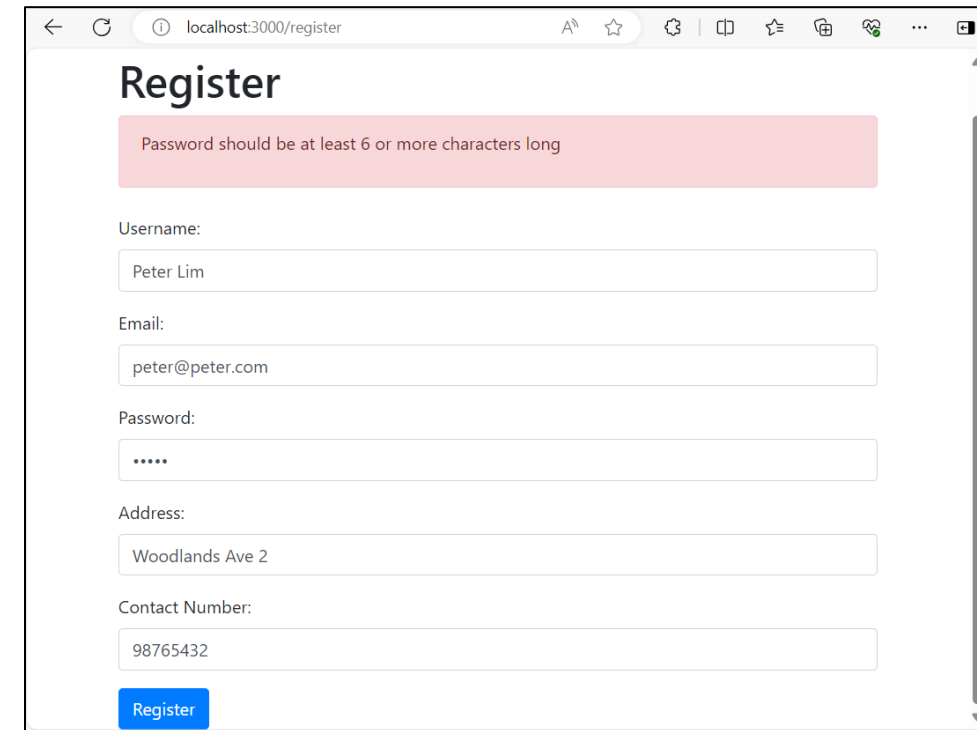
This handles user registration. It validates the input using **validateRegistration**, then inserts the new user's data into the database (hashing the password with SHA1). If successful, it flashes a success message and redirects the user to the login page.

Step 3: Test the validation

- Run your app and try to create a new user.
- Try entering a password shorter than 6 characters and see what happens.



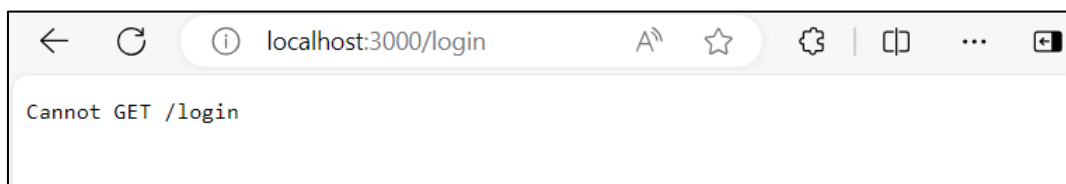
A screenshot of a web browser at localhost:3000/register. The form is titled "Register" and contains the following fields: Username (Peter Lim), Email (peter@peter.com), Password (6 dots), Address (Woodlands Ave 2), and Contact Number (98765432). A blue "Register" button is at the bottom.



A screenshot of the same web browser at localhost:3000/register. The form is titled "Register". A red error message at the top states: "Password should be at least 6 or more characters long". The form fields are: Username (Peter Lim), Email (peter@peter.com), Password (4 dots), Address (Woodlands Ave 2), and Contact Number (98765432). A blue "Register" button is at the bottom.

Step 3: Test the validation

- Run your app and try to create a new user.
- Enter all the information correctly and you should see a “**Cannot Get /login**” error. This is ok as we have not defined the **/login route** yet.



- Check your database and you should see the new user record inserted.

	id	username	email	password	address	contact
▶	1	Peter Lim	peter@peter.com	3691e5ad98c057bf6fd9ae1d79d3dc5a5d219aed	Woodlands Ave 2	98765432
★	NULL	NULL	NULL	NULL	NULL	NULL



Authentication Vs Authorisation

Authentication Vs Authorisation

Authentication	Authorisation
Who are you?	What are you allowed to access?
Login with email/password	Check user role
Creates session	Uses session role
Example: John logs in	Example: John cannot access admin page



Authentication

Authentication

- Authentication is the process of **verifying the identity of a user**.
- Involves verifying a **username and password**.
- If the login info is correct:
 - The server knows you are a real user
 - It creates a **session** so you stay logged in while using the app

Implementing Basic Authentication

- Steps to implement login function:
 1. Set up express-session for session management. (Done earlier)
 2. Create login routes.
 3. Create logout route.
 4. Test the authentication process by logging in users.

2. Create login routes.

- Open **app.js** (RegistrationApp) in Visual studio code and add in the code below:

```
/** ***** TODO: Insert code for login routes to render login page below ***** */  
app.get('/login', (req, res) => {  
  res.render('login', {  
    //retrieve success and error messages from the flash middleware and  
    //pass them to the login view for display.  
    messages: req.flash('success'),  
    errors: req.flash('error')  
  });  
});
```

2. Create login routes

- Open **app.js** (RegistrationApp) in Visual studio code and add in the code.

Get Form Data

```
const { email, password } = req.body;
```

- Retrieves Login Details submitted from form

Check Database

```
const sql = 'SELECT * FROM users WHERE email = ? AND password = SHA1(?)';
```

- checks whether the email and password match a user in the database.

Store Session

```
req.session.user = results[0];
```

- If login is successful, the first user record row returned by MySQL is stored in the session.
- E.g.

```
req.session = {
  user: {
    userId: 1,
    username: "john",
    email: "john@example.com",
    password: "5baa61e4c9b93f3f0682250b6cf8331b7ee68fd8"
  }
}
```

```

//***** TODO: Insert code for login routes for form submission below *****/
app.post('/login', (req, res) => {
  const { email, password } = req.body;
  // Validate email and password
  if (!email || !password) {
    req.flash('error', 'All fields are required.');
```

```
    return res.redirect('/login');
```

```
  }
```

```
  const sql = 'SELECT * FROM users WHERE email = ? AND password = SHA1(?)';
  db.query(sql, [email, password], (err, results) => {
    if (err) {
      throw err;
    }
    if (results.length > 0) {
      // Successful login
      req.session.user = results[0]; // store user in session
      req.flash('success', 'Login successful!');
```

```
      res.redirect('/');
```

```
    } else {
      // Invalid credentials
      req.flash('error', 'Invalid email or password.');
```

```
      res.redirect('/login');
```

```
    }
  });
});
```

Login Routes

```
const sql = 'SELECT * FROM users WHERE email = ? AND password = SHA1(?)';
db.query(sql, [email, password], (err, results) => {
  if (err) {
    throw err;
  }

  if (results.length > 0) {
    // Successful login
    req.session.user = results[0]; // store user in session
    req.flash('success', 'Login successful!');
    res.redirect('/');
  } else {
    // Invalid credentials
    req.flash('error', 'Invalid email or password');
    res.redirect('/login');
  }
});
```

app.js – /login route

```
app.get('/', (req, res) => {
  res.render('index', { user: req.session.user, messages: req.flash('success') });
});
```

app.js – / route

index.ejs

```
<% if (messages && messages.length > 0) { %>
  <div class="alert alert-success">
    <% messages.forEach(function(message) { %>
      <p><%= message %></p>
    <% }); %>
  </div>
  Hello , <%= user.username %>
  <br> <a href="/logout" class="btn btn-primary mt-3">Logout</a>
<% } else { %>
  <a href="/register" class="btn btn-primary mt-3">Register</a>
  <a href="/login" class="btn btn-primary mt-3">Login</a>
<% } %>
```

3. Create logout route.

- Open **app.js** (RegistrationApp) in Visual studio code and add in the code below:

```
/** ***** TODO: Insert code for logout route ***** **/  
app.get('/logout', (req, res) => {  
    req.session.destroy();  
    res.redirect('/');  
});
```

- **req.session.destroy();** – Ends the user session and clears all session data (e.g., login info).
- **res.redirect('/');** – Sends the user back to the home page (or another route).

Let's test it out

- Save all your files.
- Open your terminal and enter “**npx nodemon app.js**” (if your server is not already running.)
- Open your browser and enter the url <http://localhost:3000> and test the authentication process by registering, logging in and logging out users.

Sample Screenshots (Register)

Mary Tan inserted into database

Username:

Email:

Password:

Address:

Contact Number:

	id	username	email	password	address	contact
▶	1	Peter Lim	peter@peter.com	3691e5ad98c057bf6fd9ae1d79d3dc5a5d219aed	Woodlands Ave 2	98765432
	2	Mary Tan	mary@mary.com	91741b76d08f5bc1da8d3b621201812239cc732a	Tampines Ave 1	87654321
*	NULL	NULL	NULL	NULL	NULL	NULL

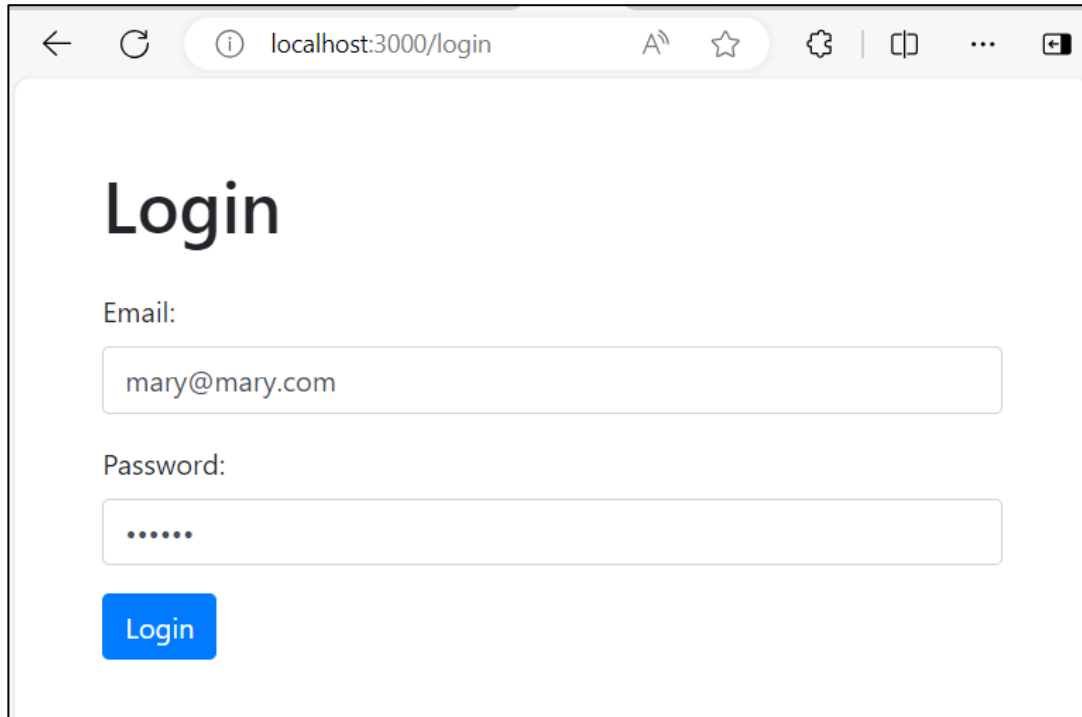
Login

Registration successful! Please log in.

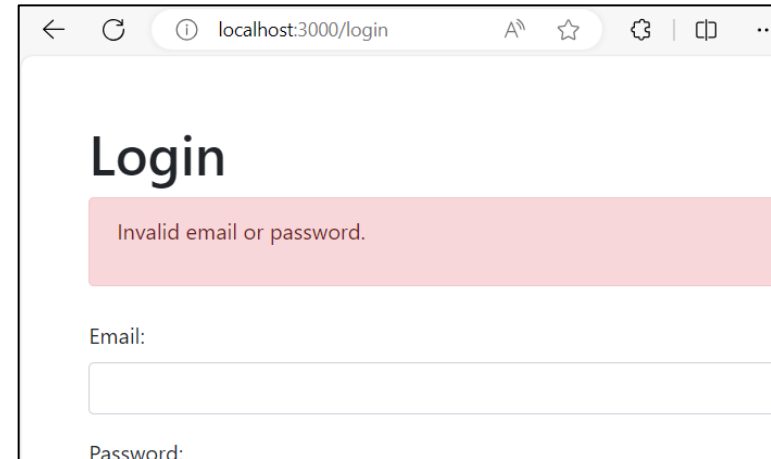
Email:

Password:

Sample Screenshots (Login)

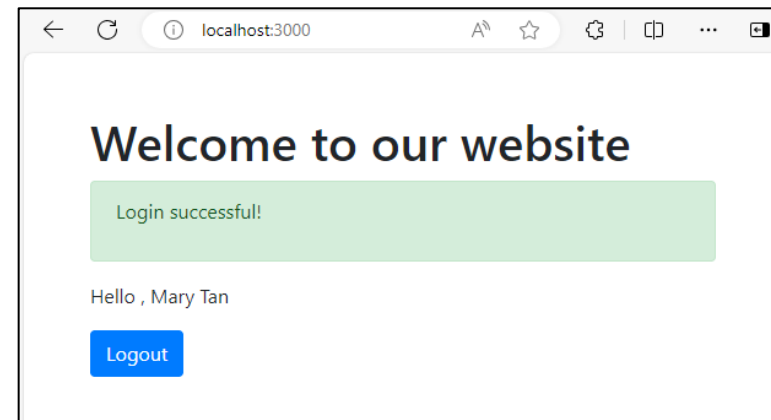


A browser window showing the login page at localhost:3000/login. The page has a title "Login". Below it, there is an "Email:" label followed by a text input field containing "mary@mary.com". Below that is a "Password:" label followed by a password input field with masked characters ".....". At the bottom left is a blue "Login" button.



A browser window showing the login page at localhost:3000/login. The page has a title "Login". Below it, there is a red error message box that says "Invalid email or password.". Below the error message are the "Email:" and "Password:" labels with their respective input fields. The "Email" field is empty.

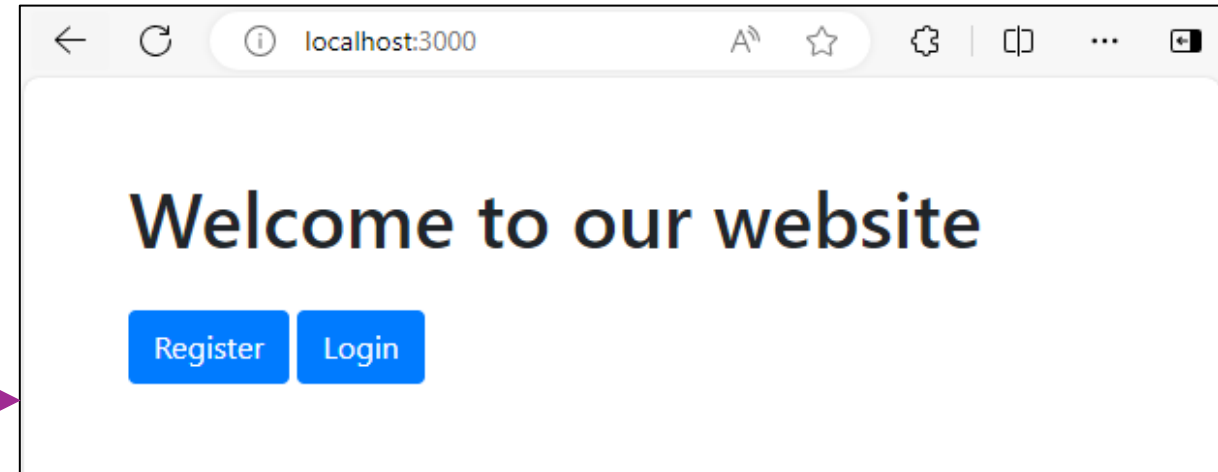
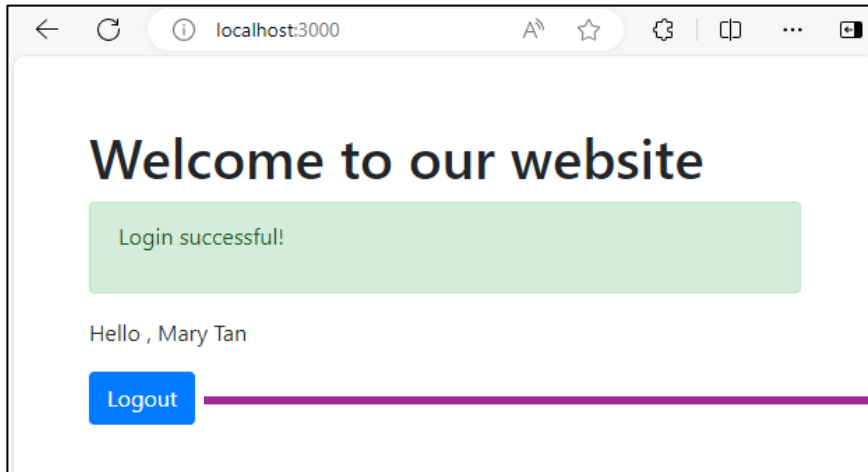
Login Failed



A browser window showing the website home page at localhost:3000. The page has a title "Welcome to our website". Below it, there is a green success message box that says "Login successful!". Below the success message, it says "Hello , Mary Tan". At the bottom left is a blue "Logout" button.

Login Success

Sample Screenshots (Logout)





Authorisation

Authorisation

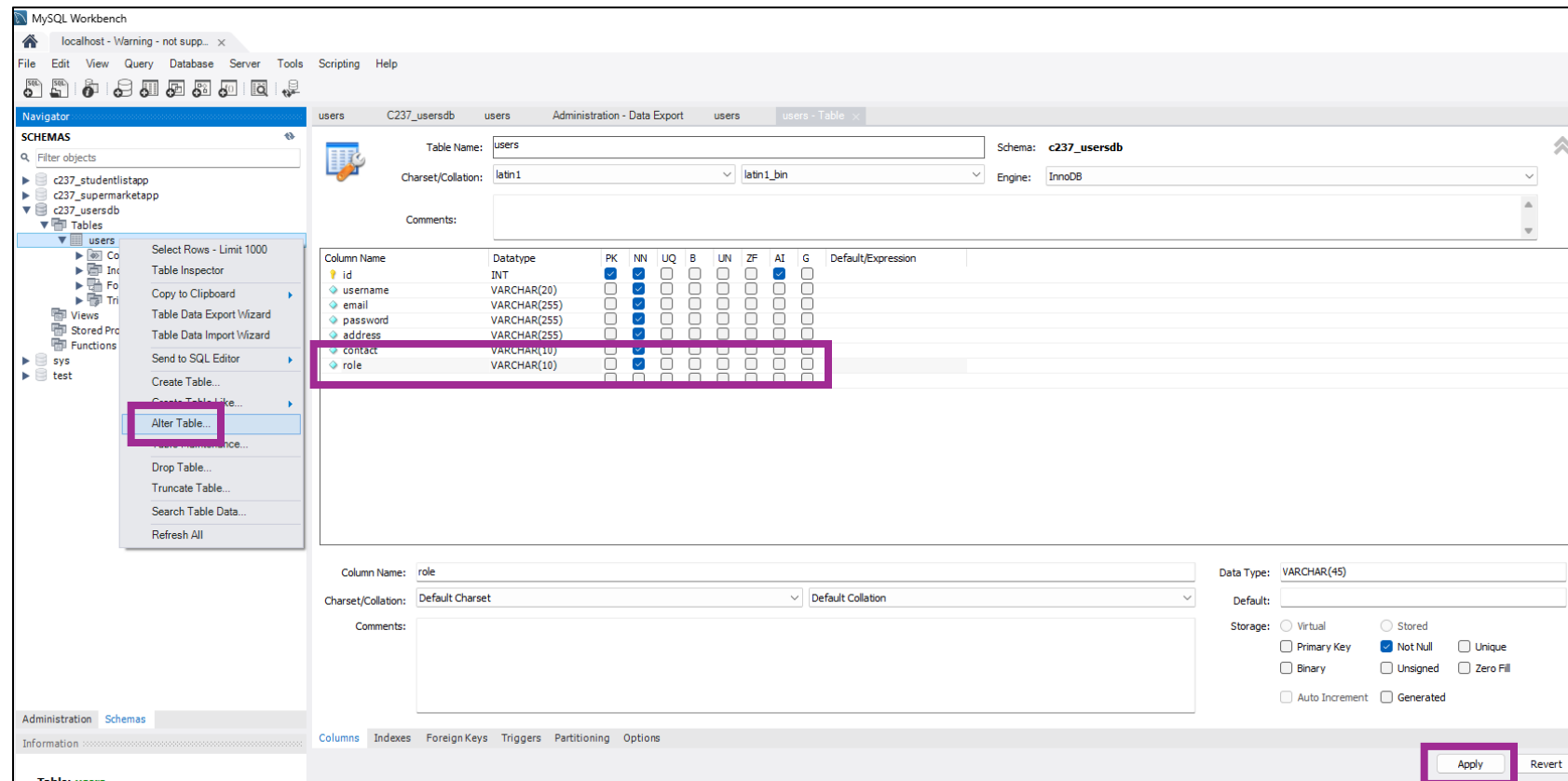
- Authorisation
 - Is the process of **granting access** to resources based on **user roles**.
 - Ensures data integrity and user experience.
- **Example: Role-Based Access Control (RBAC)**
 - Different access levels for admin and regular users.
- E.g.
 - req.session.user.role = "user";
 - Can this user access /admin?

Let's Try

- Implement role-based access control in your Registration App.
 1. Define roles and permissions
 2. Create middleware for checking authentication and roles
 3. Protect routes using middleware
 4. Test with different user roles.

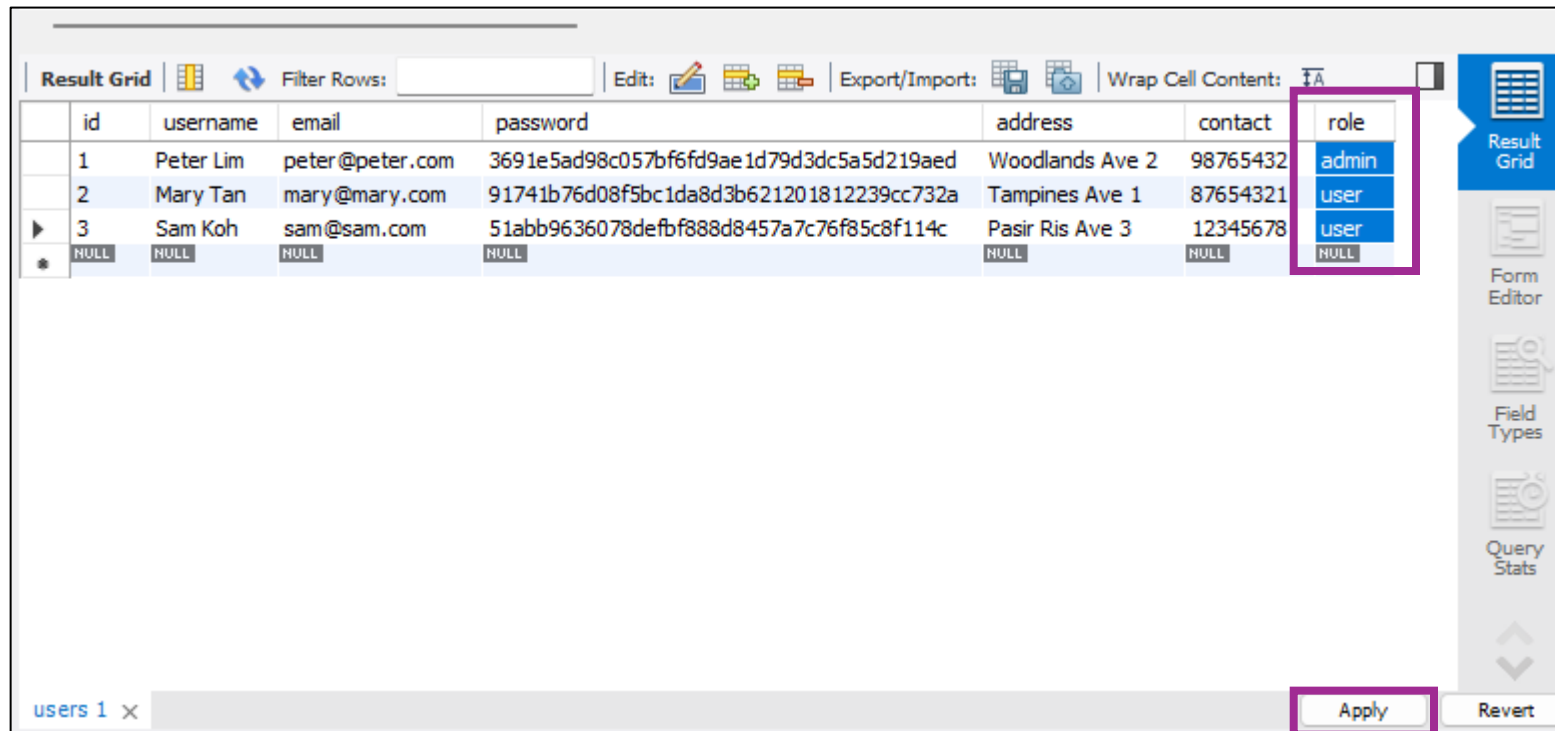
Logged in	Role	Can Access
✗	-	Login Page only
✓	User	Dashboard
✓	Admin	Dashboard + Admin Page

- **Update** your database table in MySQL Workbench:
 - Add an additional table column to your users table (Alter table -> Add role column -> Apply -> Finish)



Step 1: Define roles and permissions

- **Update** your database table in MySQL Workbench:
 - Update the roles for your users who are already in the database and click “Apply”



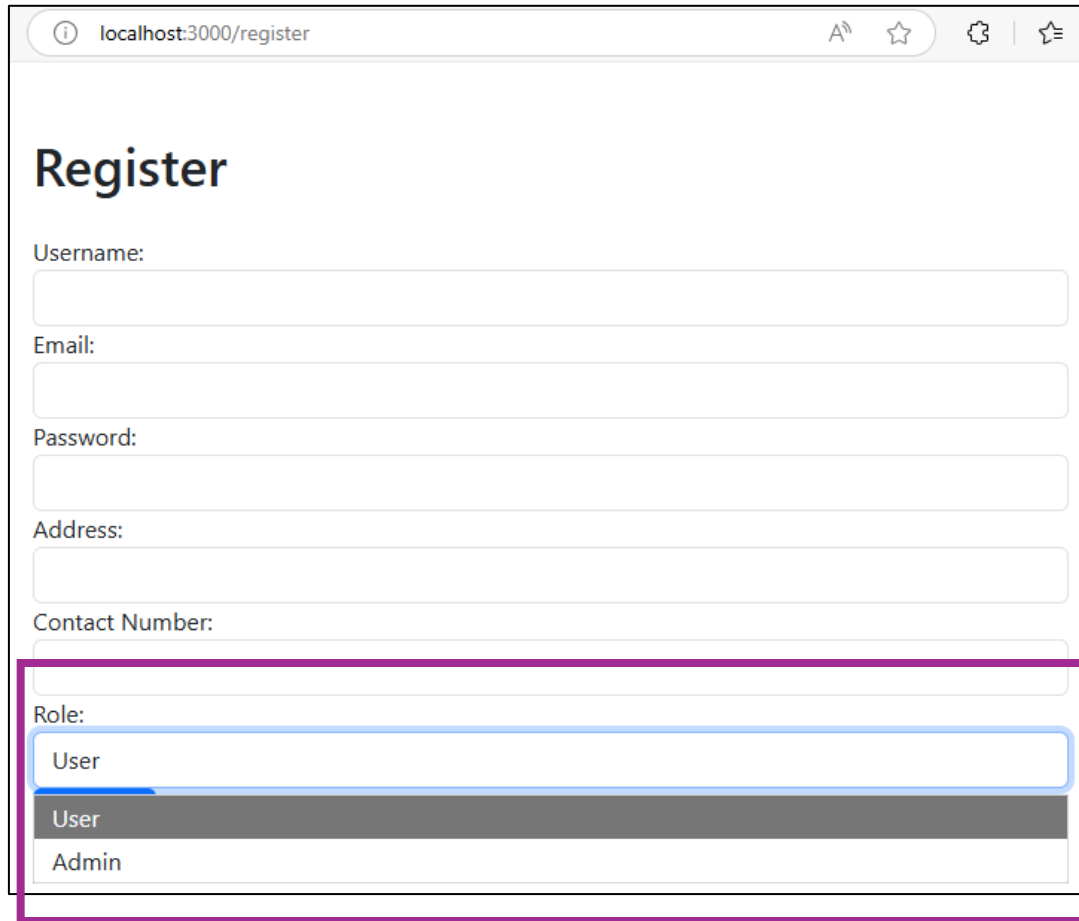
Step 1: Define roles and permissions

- Open the **registrationApp** (created in L19) in Visual Studio Code.
 - Update your registration form
 - Open your **register.ejs** file and add another select input for User/Admin

```
<div class="form-group">
  <label for="contact">Contact Number:</label>
  <input type="text" id="contact" name="contact" class="form-control" required
  <% if (formData && formData.contact) { %> value="<%= formData.contact %>" <% } else { %> value="" <% } %>>
</div>
<!-- TO DO: Add another select input for User/Admin -->
<div class="form-group">
  <label for="role">Role:</label>
  <select id="role" name="role" class="form-control" required>
    <option value="user" <% if (formData && formData.role === 'user') { %> selected <% } %>>User</option>
    <option value="admin" <% if (formData && formData.role === 'admin') { %> selected <% } %>>Admin</option>
  </select>
</div>
<button type="submit" class="btn btn-primary">Register</button>
</form>
```

Step 1: Define roles and permissions

- Run your app. Your updated register form should look like this:



localhost:3000/register

Register

Username:

Email:

Password:

Address:

Contact Number:

Role:

User

User

Admin

Step 1: Define roles and permissions

- Open **app.js** (RegistrationApp) in Visual studio code and update the register route:

```
//***** Integrate validateRegistration into the register route. *****/
app.post('/register', validateRegistration, (req, res) => {

  //***** TO DO: Update register route to include role *****/
  const { username, email, password, address, contact, role } = req.body;

  const sql = 'INSERT INTO users (username, email, password, address, contact, role) VALUES (?, ?, SHA1(?), ?, ?, ?)';
  db.query(sql, [username, email, password, address, contact, role], (err, result) => {
    if (err) {
      throw err;
    }
    console.log(result);
    req.flash('success', 'Registration successful! Please log in.');
    res.redirect('/login');
  });
});
```

Step 2: Create middleware for checking authentication and roles

- Open **app.js** (RegistrationApp) in Visual studio code and add in (near the top of app.js page) the code below:

```
/** ***** TO DO: Create a Middleware to check if user is logged in ***** **/  
const checkAuthenticated = (req, res, next) => {  
  if (req.session.user) {  
    return next();  
  } else {  
    req.flash('error', 'Please log in to view this resource');  
    res.redirect('/login');  
  }  
};
```

Step 2: Create middleware for checking authentication and roles

- Open **app.js** (RegistrationApp) in Visual studio code and add in the code below:

```
/** ***** TO DO: Create a Middleware to check if user is admin ***** **/  
const checkAdmin = (req, res, next) => {  
  if (req.session.user.role === 'admin') {  
    return next();  
  } else {  
    req.flash('error', 'Access denied');  
    res.redirect('/dashboard');  
  }  
};
```

Step 3: Protect routes using middleware

- Open **app.js** (RegistrationApp) in Visual studio code and add in the code below:

```
//***** TO DO: Insert code for dashboard route to render dashboard page for users *****/  
app.get('/dashboard', checkAuthenticated, (req, res) => {  
    res.render('dashboard', { user: req.session.user });  
});  
  
//***** TO DO: Insert code for admin route to render dashboard page for admin *****/  
app.get('/admin', checkAuthenticated, checkAdmin, (req, res) => {  
    res.render('admin', { user: req.session.user });  
});
```

Step 3: Protect routes using middleware

- Open **app.js** (RegistrationApp) in Visual studio code and **update** the /login route:

```
if (results.length > 0) {  
  // Successful login  
  req.session.user = results[0]; // store user in session  
  req.flash('success', 'Login successful!');  
  //***** TO DO: Update to redirect users to /dashboard route upon successful log in *****//  
  res.redirect('/dashboard');  
} else {  
  // Invalid credentials  
  req.flash('error', 'Invalid email or password.');
```

Let's Test it out!

- Save all your files.
- Open your terminal and enter “**npx nodemon app.js**” (if your server is not already running.)
- Open your browser and enter the url <http://localhost:3000> and test the authentication process by registering and logging in users.

Sample Screenshots (Register)

Register

Username:

Email:

Password:

Address:

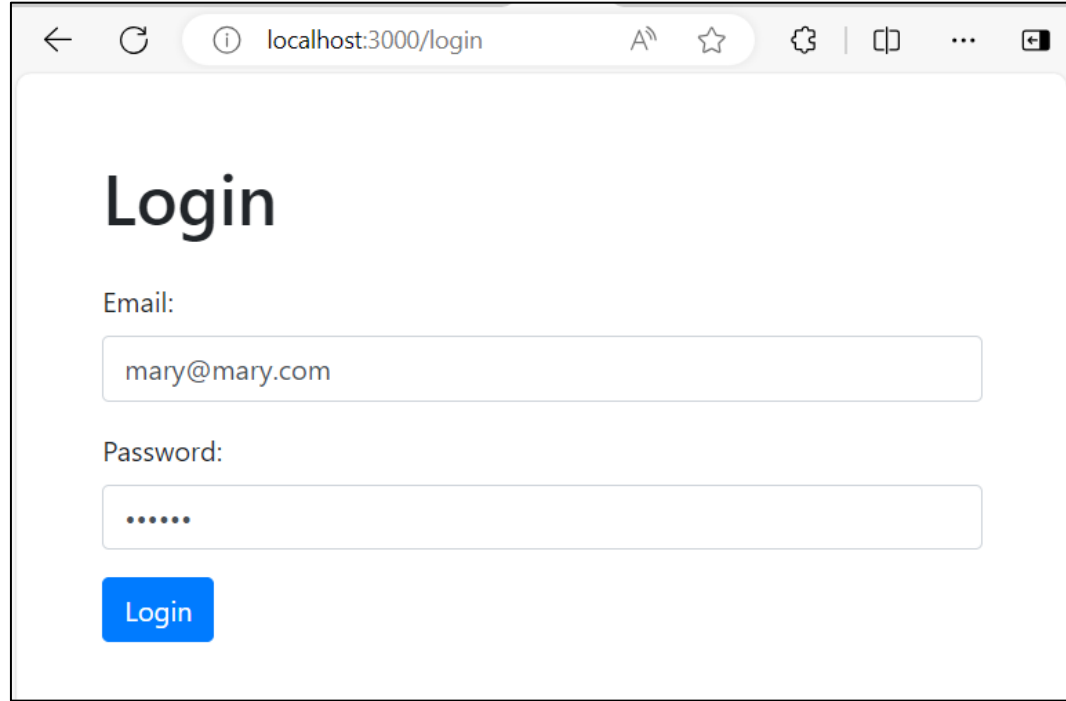
Contact Number:

Role:

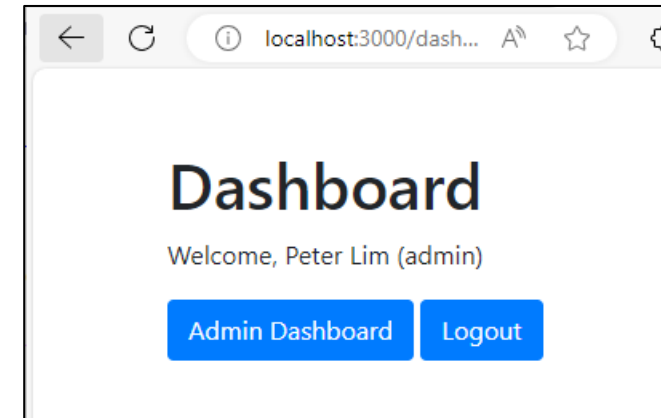
	id	username	email	password	address	contact	role
▶	1	Peter Lim	peter@peter.com	3691e5ad98c057bf6fd9ae1d79d3dc5a5d219aed	Woodlands Ave 2	98765432	admin
	2	Mary Tan	mary@mary.com	91741b76d08f5bc1da8d3b621201812239cc732a	Tampines Ave 1	87654321	user
	3	Sam	sam@sam.com	51abb9636078defbf888d8457a7c76f85c8f114c	Pasir Ris Ave 3	12345678	user
	4	Paul Koh	paul@paul.com	226cffca4538306166e3c75d4e16e4cbc981379b	Bedok Ave 4	23456789	admin

Inserted into database

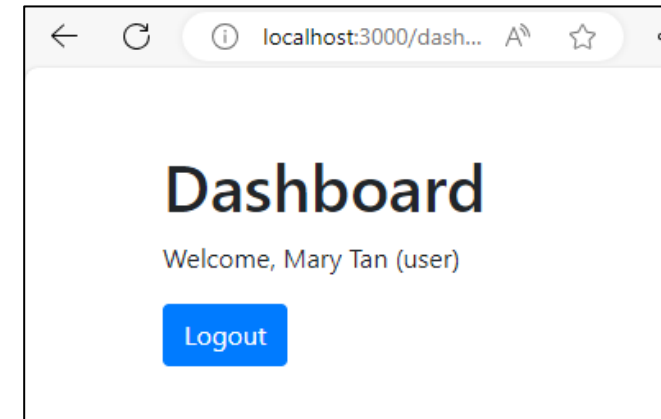
Sample Screenshots (Login)



A screenshot of a web browser showing a login page. The address bar displays 'localhost:3000/login'. The page has a large 'Login' heading. Below it, there are two input fields: 'Email:' with the value 'mary@mary.com' and 'Password:' with masked characters '.....'. A blue 'Login' button is positioned at the bottom left of the form.

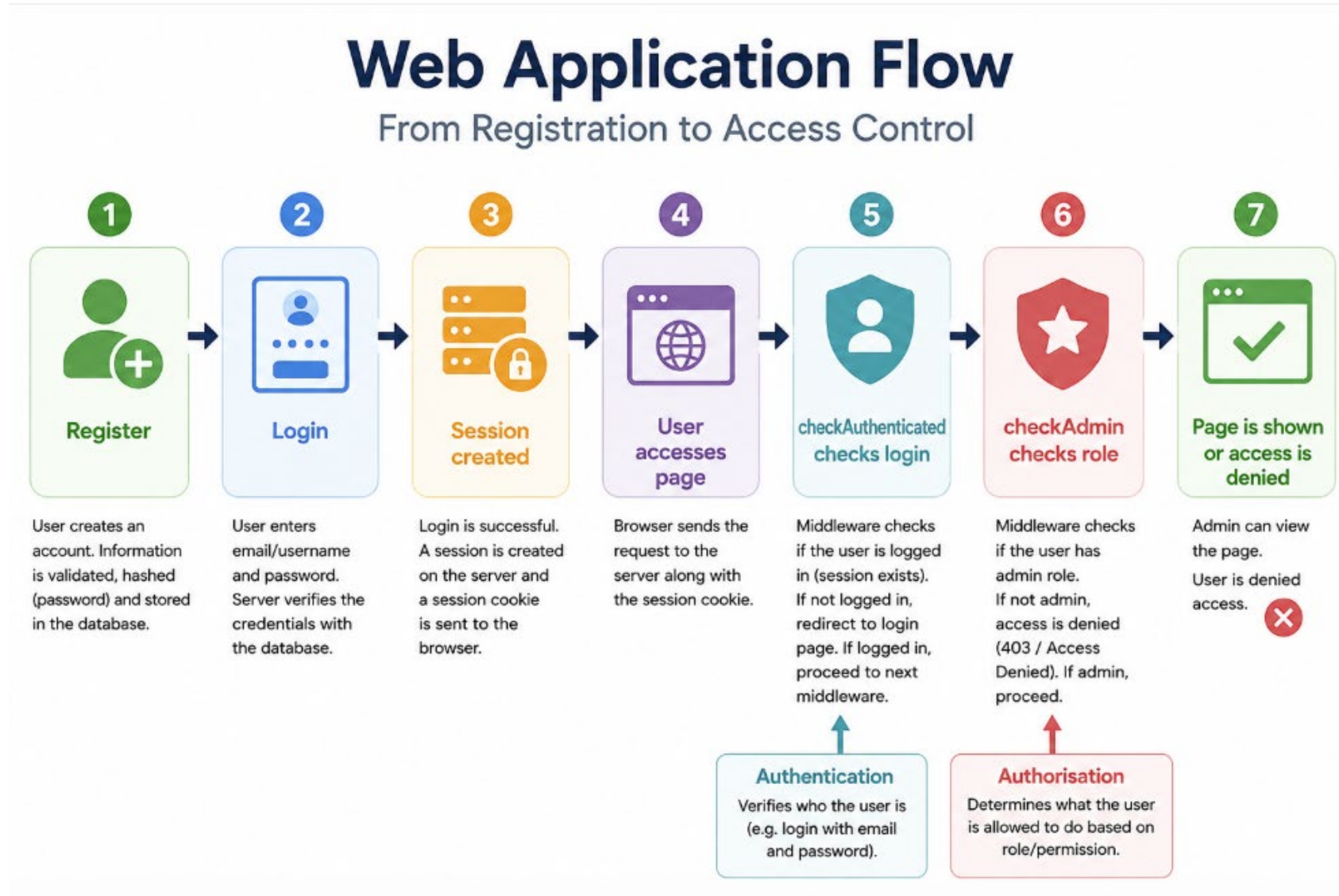


Admin Login



User Login

What have you learnt?



End of Lesson 19